
Time

Time

- ❑ The “real”, “global”, “physical” time
 - ❑ The time on the computer (computer clock)
 - ❑ Logical time
-
- ❑ Why do we need a concept of time?

Hardware and Software Clocks

□ At “real” time t , the computer’s hardware clock is $H(t)$

□ The OS calculates the time on the computer’s software clock

$$C(t) = \alpha H(t) + \beta$$

★ E.g. a 64 bit number giving nanoseconds since some base time

□ Clock resolution

★ Period between update of clock value

★ How accurate does it need to be?

Hardware and Software Clocks

□ Clock Drift:

- ★ Rate at which clocks deviates from real time
- ★ Average: 1 sec drift in 11 days
- ★ How to adjust a software clock when one realizes that it is slow (shows an outdated time) or fast (shows future time)?

□ Monotonicity

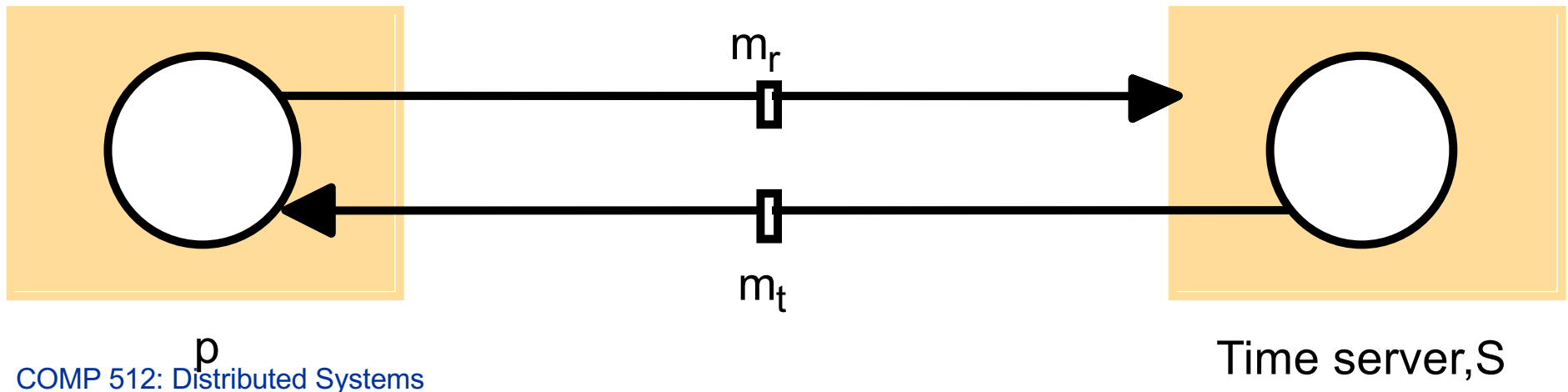
- ★ $t' > t \Rightarrow C(t') \geq C(t)$
- ★ can achieve monotonicity with a hardware clock that runs fast by adjusting the values of α and β in $C_i(t) = \alpha H_i(t) + \beta$

Synchronization

- ❑ Internal synchronization within bound D
 - ☆ given a set of computer clocks $C_i(t)$, $1 \leq i \leq N$
 - ☆ For each $C_i(t)$, $C_j(t)$, $1 \leq i, j \leq N$: $|C_i(t) - C_j(t)| < D$
- ❑ Coordinated Universal Time (UTC)
 - ☆ based on very accurate physical clocks
 - ☆ Computer need receivers;
 - 0.1-10 milliseconds accurate for land-based reception
 - 1 microsecond through GPS
- ❑ External synchronization within bound D
 - ☆ Given a source S of UTC time and computer clock C
 - ☆ $|S(t) - C(t)| < D$

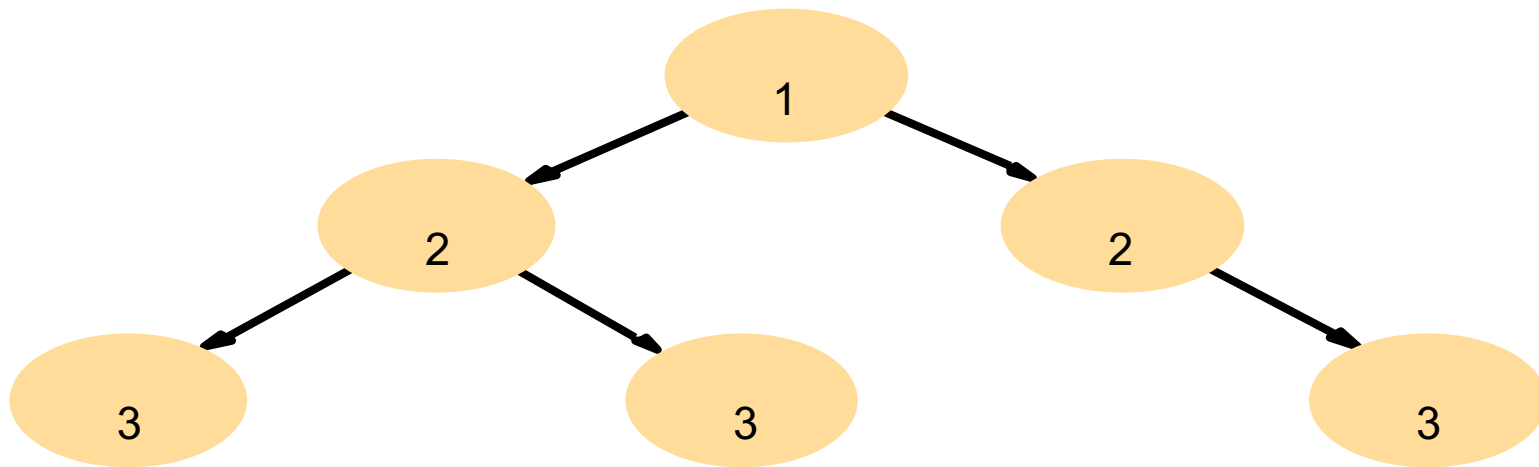
Synchronizing clocks in an asynchronous system

- ❑ Idea: use a time server S that receives signals from a UTC source
- ❑ Synchronizing process p with time server
- ❑ Build average over several requests



Network Time Protocol NTP

- ❑ Primary Servers are connected to UTC sources
- ❑ Secondary servers are synchronized to primary servers
- ❑ lowest level servers are user's computers
- ❑ accuracy of
 - ★ 10s of milliseconds for Internet
 - ★ 200 microseconds in Intranet



What time would it be if an Elephant sat
on the watch?

*It sure would be a time to buy a new
watch.*

Events and Logical time

- ❑ A distributed system consists of a collection of N processes p_i , $i=1, 2, 3, \dots, N$
- ❑ Information exchange only through message exchange
- ❑ Each process p_i has a state s_i . The state changes according to the operations the process performs.
 - ☆ State: content of all application variables.
- ❑ Each process executes a sequence of events/actions,
 - ☆ send, receive or change own state
 - ☆ (ignore multi-threading, multi-processor concurrency)

Events and Logical time

□ Ordering of events in a distributed system

☆ The sequence of events within a single process can be placed in a total order:

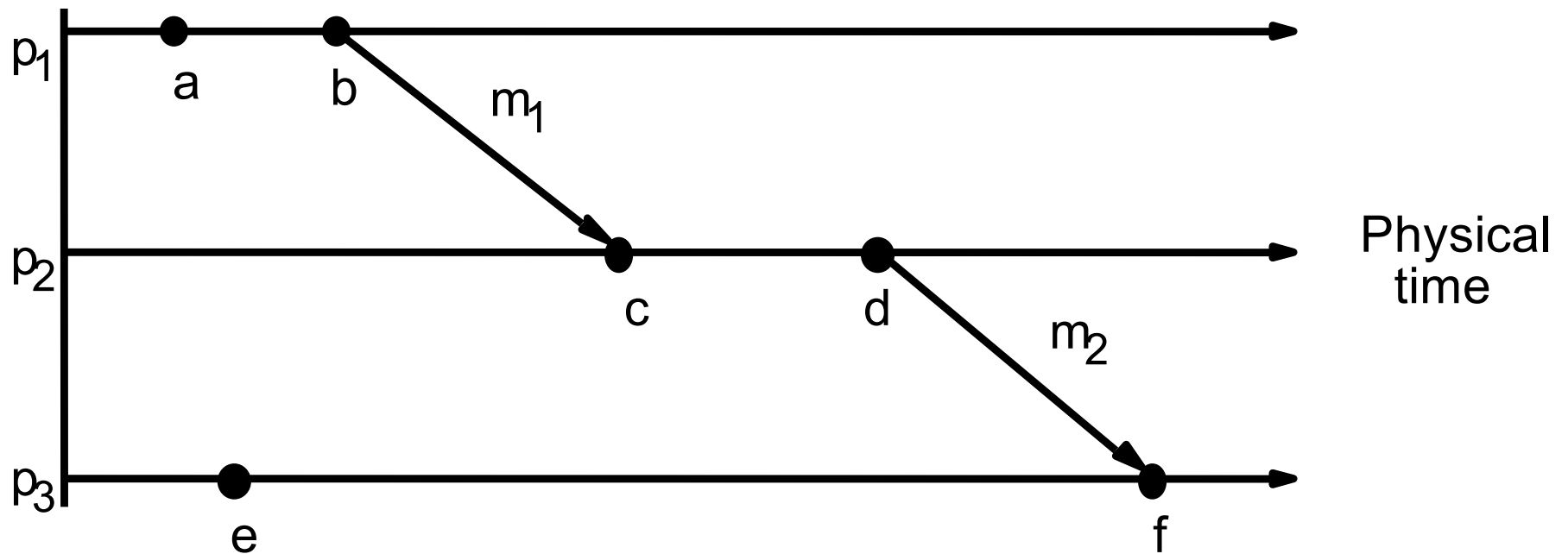
- $e \rightarrow_i e'$ if and only if event e occurs before e' at p_i .
- Whenever a message is sent, the event of sending a message occurs before the event of receiving the message

□ A history of process p_i is a series of events ordered by

$\rightarrow_i$

$history(p_i) = h_i = \langle e_i^0, e_i^1, e_i^2, \dots \rangle$

Events in a distributed system



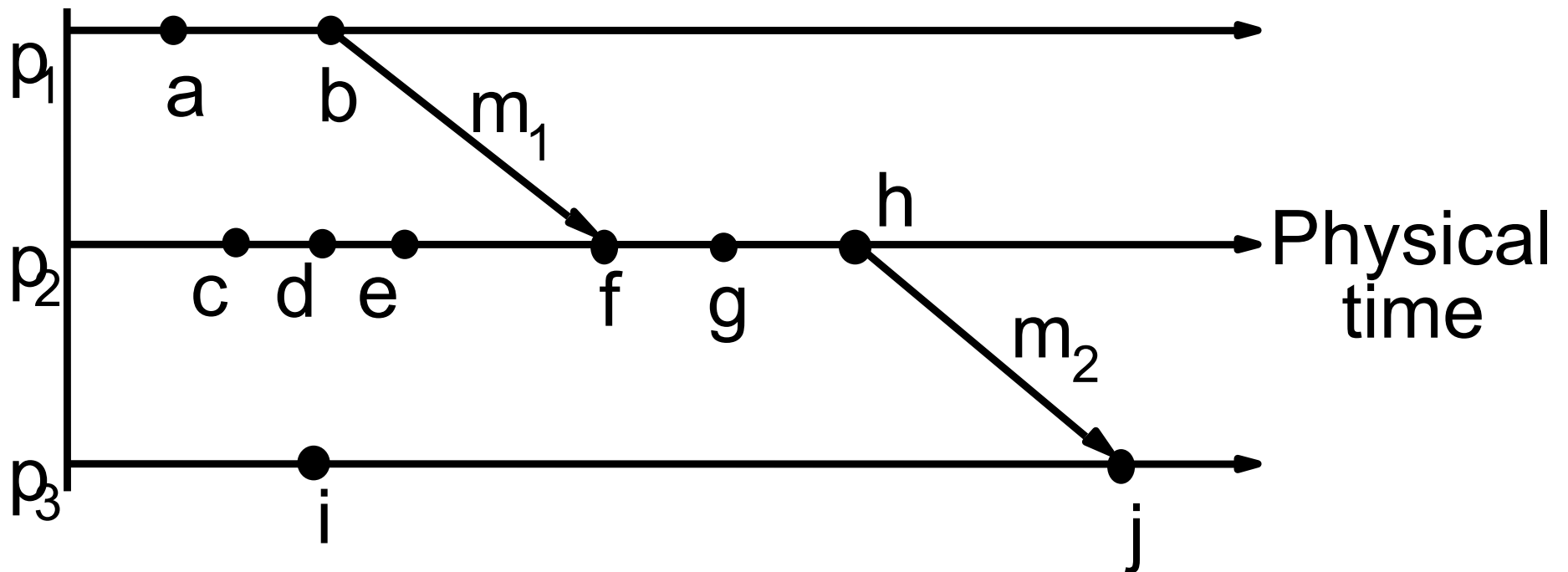
Logical time

- We define the happened-before relation, denoted by $\rightarrow$, as follows
 - ★ If there is a process p_i : $e \rightarrow_i e'$, then $e \rightarrow e'$
 - ★ For any message m , $\text{send}(m) \rightarrow \text{receive}(m)$ (where $\text{send}(m)$ is the event of sending the message and $\text{receive}(m)$ the receiving event)
 - ★ If e , e' , and e'' are events such that $e \rightarrow e'$ and $e' \rightarrow e''$, then $e \rightarrow e''$
 - ★ If neither $e \rightarrow e'$ nor $e' \rightarrow e$, then e and e' are concurrent $e \parallel e'$

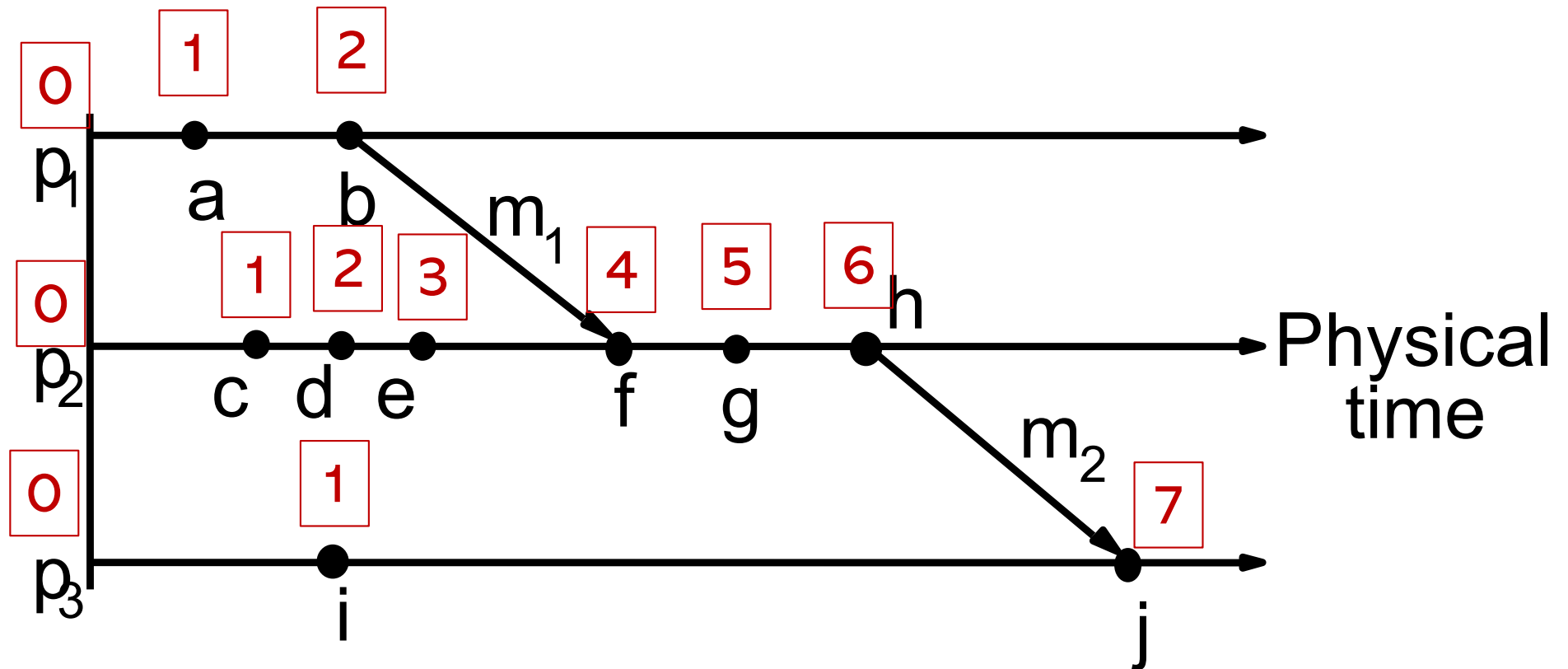
Logical clocks (Lamport clocks)

- ❑ Each process p_i maintains a logical clock L_i .
- ❑ L_i is a monotonically increasing counter (no relationship to real time)
- ❑ Each site timestamps events with the value of its logical clock.
- ❑ We denote the timestamp of event e at p_i by $L_i(e)$ (and omit the index if we speak about any event in the system)
- ❑ Goal: whenever $e \rightarrow e'$, then $L(e) < L(e')$, i.e., when e happens before e' , then e should have a smaller timestamp than e' .

Events in a distributed system



Events in a distributed system



Logical clocks (Lamport clocks)

□ Algorithm at process p_i when event e takes place:

$L_i := L_i + 1$

If Local event e :

$L_i(e) := L_i$

Else if Send(m) event e :

$L_i(e) := L_i;$

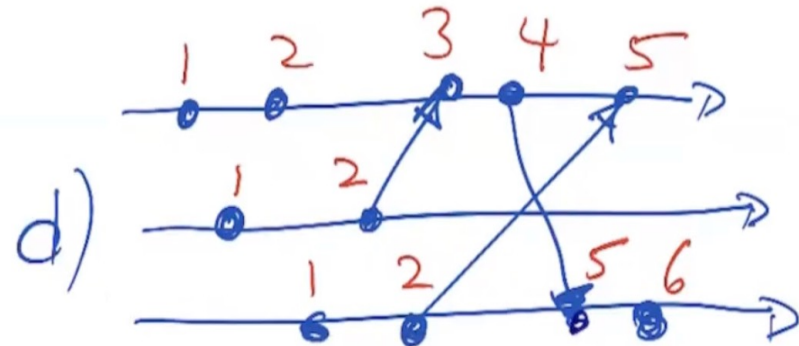
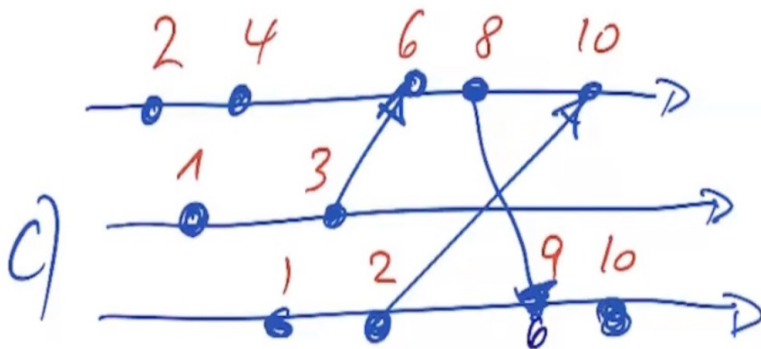
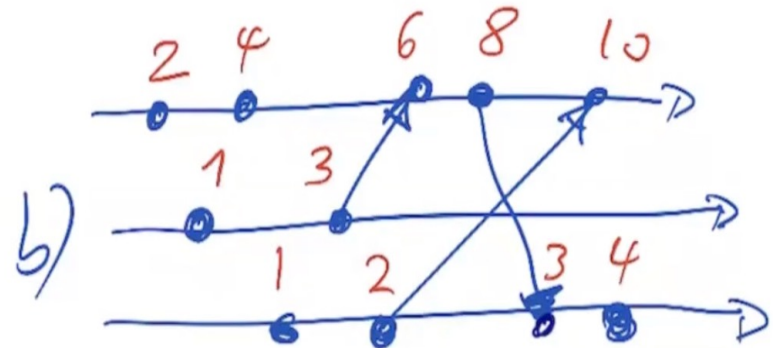
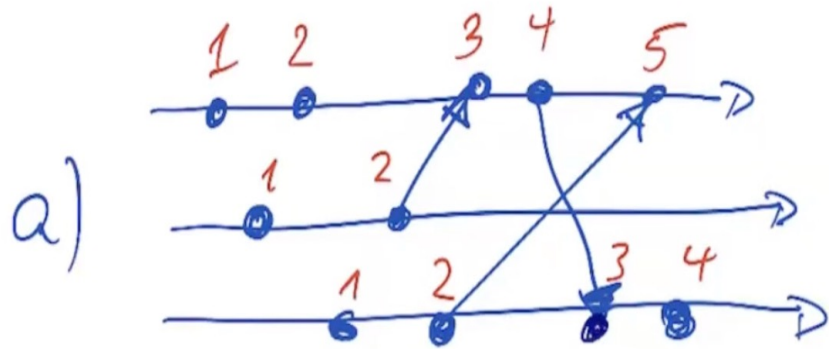
Piggyback $L_i(e)$ on m ($m, L_i(e)$)

Else if Receive($m, L_j(e')$) event e :

$L_i := \max(L_i, L_j(e') + 1);$

$L_i(e) := L_i$

Which ones are correct?



Timestamp ordering

- With Lamport clocks it is easy to see that
 - ★ If $e \rightarrow e'$ then $L(e) < L(e')$
- The opposite does not hold
 - ★ If $L(e) < L(e')$, then this does not mean that $e \rightarrow e'$, instead it could also be that $e \parallel e'$
- A challenge is that two events that happen one after the other in one process, can both be concurrent to an event in a different process.
- Therefore, a timestamping scheme that provides total order does not work!

Vector Clocks

□ Goal: find a timestamp V such that

★ $e \rightarrow e' \iff V(e) < V(e')$

★ In particular, this means, that if neither $e \rightarrow e'$ nor $e' \rightarrow e$, then neither $V(e) < V(e')$ nor $V(e') < V(e)$ (timestamps are incomparable)

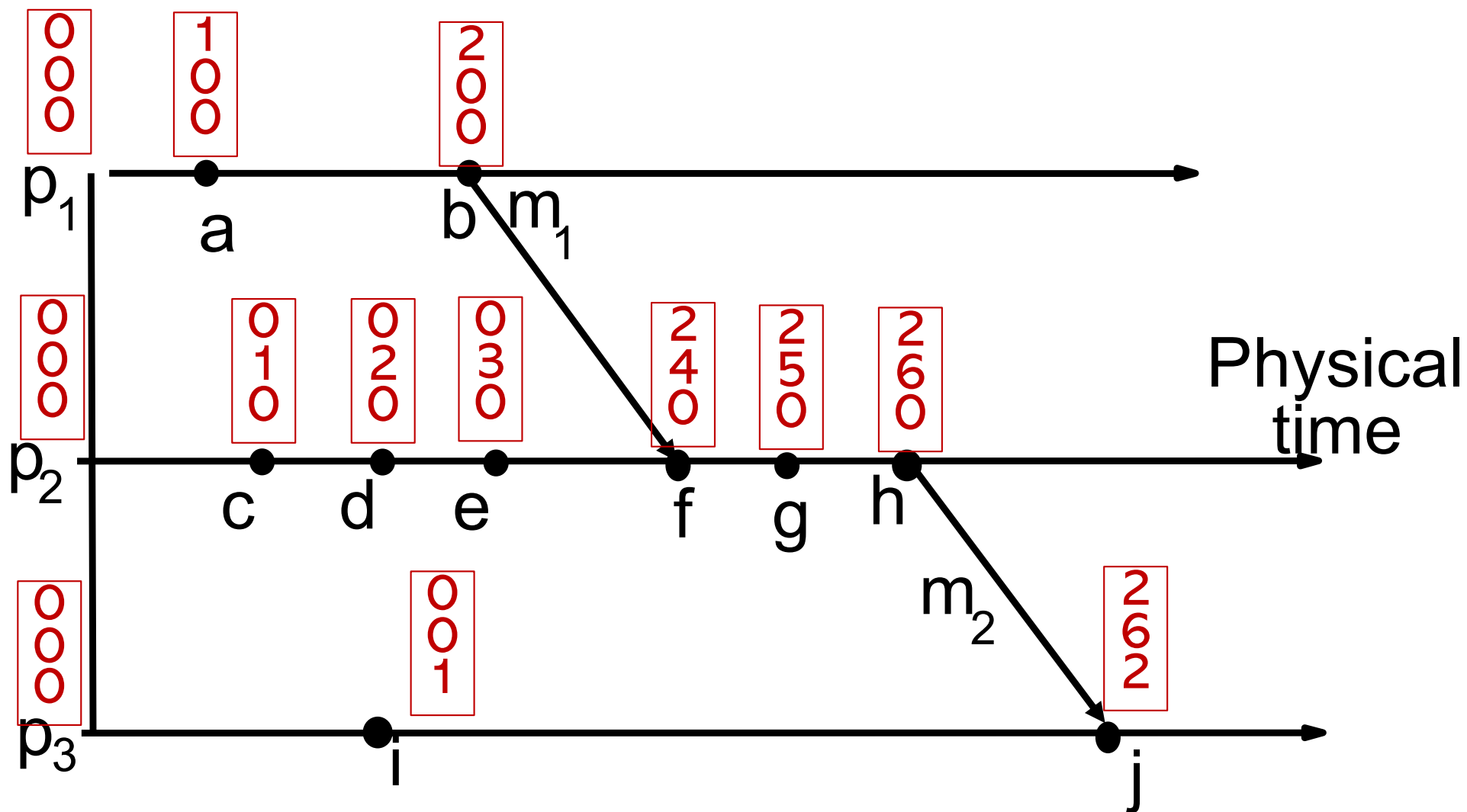
□ Each processor p_i has a vector clock V_i which is an array of N integers. (one for each processor)

□ How to compare vector timestamps?

★ $V = V'$ iff $V[j] = V'[j]$ for $j = 1, 2, \dots, N$

★ $V \leq V'$ iff $V[j] \leq V'[j]$ for $j = 1, 2, \dots, N$

★ $V < V'$ iff $V \leq V'$ and not $V = V'$



Algorithm at process p_i

Upon start:

$V_i[j] := 0$, for $j = 1, 2, \dots, N$

Upon local event e :

$V_i[i] := V_i[i] + 1$;

$V_i(e) := V_i$

Upon event e to send m :

$V_i[i] := V_i[i] + 1$;

$V_i(e) := V_i$

Send (m , V_i)

Upon receive(m, t) event e :

$V_i[i] := V_i[i] + 1$;

$V_i[j] := \max(V_i[j], t[j])$, for $j = 1, 2, \dots, N$. (componentwise maximum)

Vector timestamps

